LABORATORIO S09-Guerrero López Uriel Ivan

Que el aviso no tumbe la transferencia

Bitácora de implementación · Mexi Banco · Sistemas Distribuidos 2027-1

Objetivo. Separar la transferencia del servicio de notificaciones con RabbitMQ, y demostrar que el consumidor puede recuperarse de caídas, duplicados y mensajes inválidos.

Contrato aplicado. Exchange topic durable transferencias; claves transferencia.realizada y transferencia.revertida; cola durable notificacion.transferencias; DLX y DLQ según el contrato.

Parte 0 · El problema

Al apagar notificacion y ejecutar transferencias con la implementación síncrona inicial, la respuesta falló y las filas 18 y 19 quedaron en estado ABONADA. En la evidencia, el saldo de origen quedó en $913 y el de destino en $587 al cierre de esa reproducción, en la que se realizaron dos abonos de $2.

En una frase: la saga dependía de una llamada HTTP síncrona al servicio de avisos y el error de ese servicio interrumpía la saga después de mover el dinero.



Captura 1. Estado ABONADA en las dos transferencias fallidas al notificar.



Captura 2. Saldo de origen: $913.



Captura 3. Saldo de destino: $587.

Parte 1 · Publicar el evento

Cuando la saga termina, TransferenciaService crea un evento con identificador estable y lo publica en el exchange transferencias usando la routing key correspondiente al resultado. El evento lleva el tipo, identificador de transferencia, CLABEs, monto, saldos y fecha. Quité la llamada HTTP directa a notificacion, por lo que el resultado de esa segunda aplicación ya no detiene la saga.

El publisher confirm informa que RabbitMQ aceptó la publicación en el broker. En combinación con la publicación persistente, sirve para detectar que el broker recibió el mensaje. No garantiza que un consumidor lo procese, que se guarde un aviso ni que la operación de negocio y el mensaje se confirmen como una sola transacción; sin Outbox queda la ventana de doble escritura si RabbitMQ está caído al publicar.



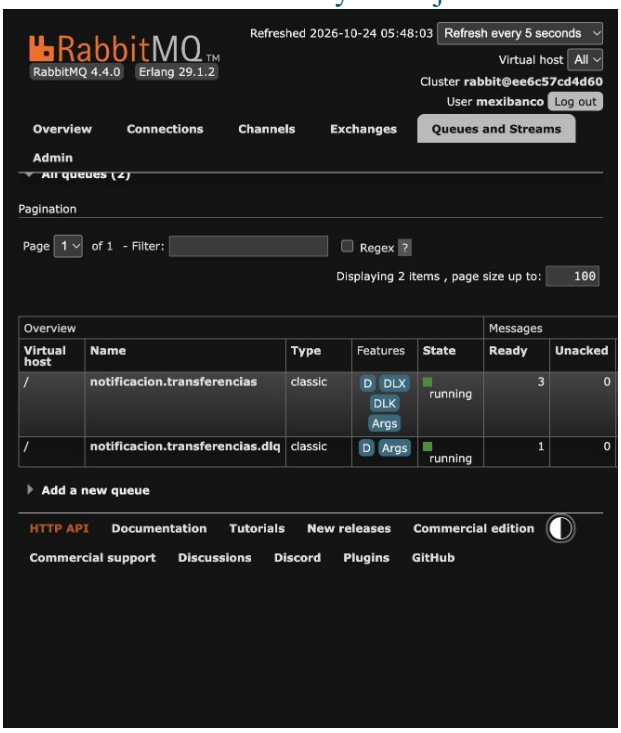
Captura 4. Panel RabbitMQ: exchange transferencias de tipo topic y durable.

El consumidor declara la cola durable y la enlaza con transferencia.*. Usa ack manual y confirma hasta terminar el guardado. Para una transferencia realizada genera avisos para origen y destino; para una revertida informa al origen.

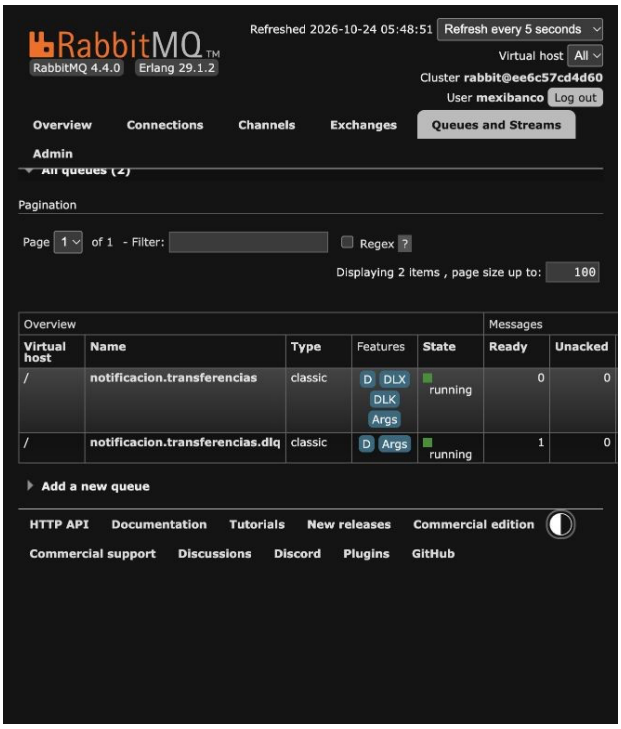
Parte 2 · Desacople en tiempo

Con notificacion detenido hice tres transferencias de $10. El exchange y la cola conservaron los mensajes mientras el consumidor estaba fuera. Al volver a encenderlo, la cola se vació y los avisos aparecieron en las bandejas consultadas por
CLABE.

Parte 2 · Evidencia de la cola y bandejas

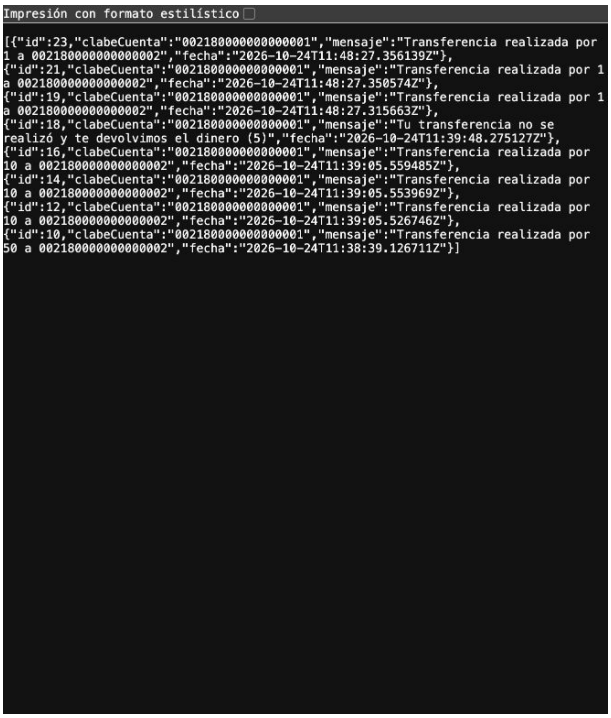


Captura 5. Cola principal con 3 Ready mientras notificacion está apagado.

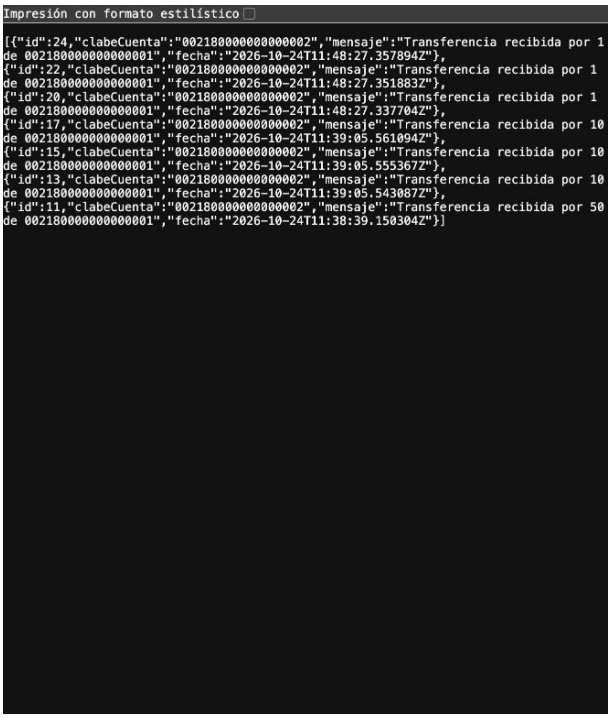


Captura 6. Después: cola principal en 0 Ready; el mensaje inválido se ve en la DLQ.

La bandeja recibió los avisos una vez que el consumidor estuvo disponible. Esto muestra el desacople en tiempo:
productor y consumidor no necesitan estar vivos al mismo tiempo para conservar y procesar el evento.
También probé una transferencia con falla provocada de abono; al compensarse, se publica TransferenciaRevertida y se registra el aviso de devolución al origen.



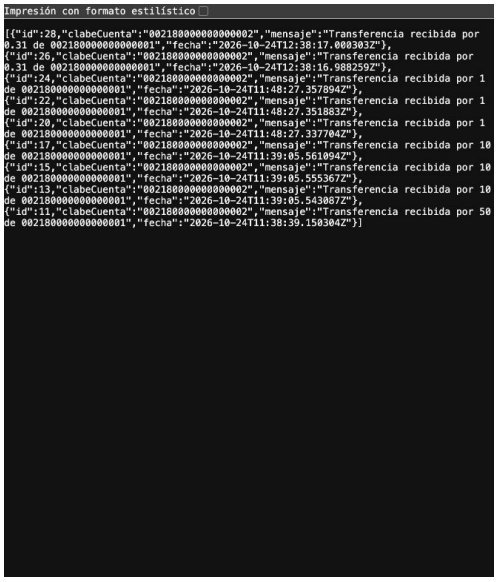
Captura 7. Avisos para la CLABE de origen.



Captura 8. Avisos para la CLABE de destino.

Parte 3 · El duplicado y la idempotencia

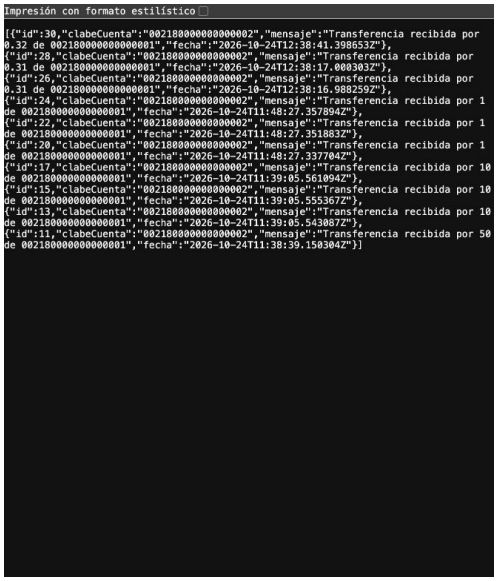
Publiqué dos veces el mismo evento con el mismo eventoId. En el primer experimento quité temporalmente la comprobación de duplicados: la bandeja mostró dos avisos iguales de $0.31 para el mismo evento.



Captura 9. Antes del arreglo: dos notificaciones repetidas para el mismo eventoId.

El arreglo guarda el eventoId en la tabla de eventos procesados, que tiene clave única. Esa inserción y la creación de avisos ocurren en la misma transacción. Si el identificador ya existe, el consumidor no vuelve

a crear avisos y confirma el mensaje. Así, si RabbitMQ reentrega después de que el guardado terminó pero antes del ack, el mismo evento no genera un segundo juego.



Captura 10. Después: para el evento repetido de $0.32 aparece un solo aviso en la bandeja consultada.

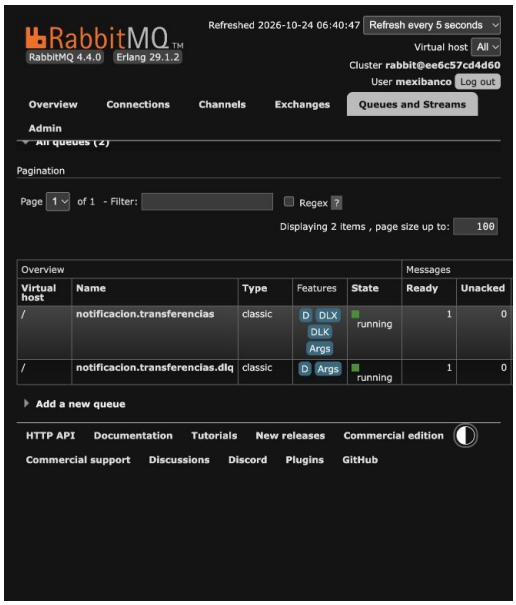
La entrega sigue siendo al menos una vez; la aplicación hace idempotente el efecto persistido. El ack por sí solo no evita duplicados.

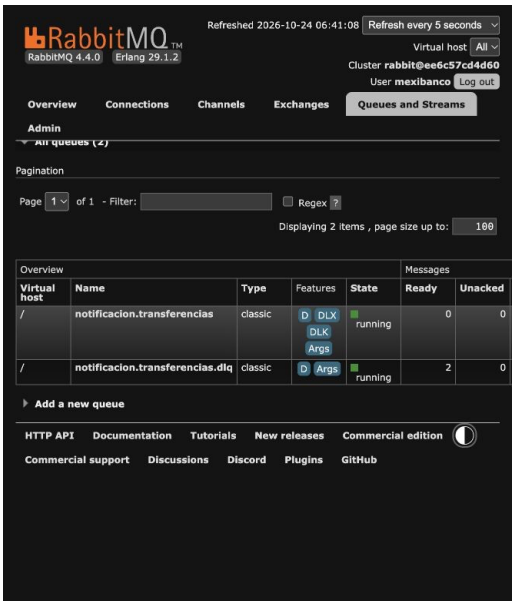
Parte 4 · Mensaje envenenado y DLQ

Publiqué un cuerpo JSON inválido en transferencia.realizada. El consumidor no puede convertirlo en un evento válido. Configuré la cola principal con x-dead-letter-exchange y x-dead-letter-routing-key; el consumidor hace nack sin reencolar cuando el mensaje no se puede leer. RabbitMQ lo enruta al exchange transferencias.dlx y a notificacion.transferencias.dlq.

Con el consumidor detenido, la captura previa muestra el mensaje malo en la cola principal (Ready=1). Al arrancarlo, el consumidor lo rechaza sin reencolar; la captura posterior muestra la cola principal en 0 y el mensaje trasladado a la DLQ.

Las transferencias válidas siguen drenándose normalmente.





Captura 11. Antes: mensaje malo en la cola principal, Ready=1.Captura 12. Después: principal Ready=0 y DLQ Ready=2.

Preguntas de cierre

1. RabbitMQ caído al completar. En esta versión sin Outbox, la transferencia puede completar y el evento no llegar a publicarse; el aviso puede perderse. El publisher confirm no lo arregla porque solo confirma publicaciones que alcanzan el broker: no guarda el evento junto con el cambio de estado de la base de datos.

2. Orden del ack. El ack va después de guardar. Si confirmo antes y el proceso falla antes del guardado, RabbitMQ elimina el mensaje y se pierde el aviso. Si guardo primero y fallo antes del ack, RabbitMQ puede reentregarlo; por eso guardo el eventoId con restricción única y el efecto es idempotente.

3. Elección y sacrificio. Elijo el procesamiento asíncrono porque el aviso no debe tumbar la transferencia. Gano tolerancia a que notificacion esté temporalmente fuera. Sacrifico la confirmación inmediata del aviso y, sin Outbox, la garantía de publicar si RabbitMQ cae justo en ese momento.